00 ACTION 11A / CLASSES IN UNITY

Classes in Unity

Create three C# classes that move, rotate, and recolor Unity objects. Trace each visible change back to its data, properties, and methods.

The Training Ring needs a moving marker, a rotating signal, and a colored indicator. You will build each behavior separately, then attach all three to one object.

> **Start here:** Read the class example below, then follow the scene setup. Use the instructor's opening demonstration, your team, and the references as you work. Bring the exact error and the line you are investigating when you need help.

Action 11A · 10 points · Complete and submit the same day: Monday, November 2, 2026, before lab ends at 12:35 PM Pacific. Canvas records the official deadline and grade.

01 C# FOUNDATIONS

Understand classes & properties

A **class** defines a type: what data its objects hold and what actions they can perform. An **instance** is one object of that type. If two cubes each have a `TranslateObject` component, there are two instances with separate speed values.

Class

`TranslateObject`

One definition of a movement behavior.

Instance A

MoveCube's component

Speed: 1 unit/second

Instance B

Another cube's component

Speed: 2 units/second

A Unity **GameObject** is a scene object that holds components. Its **Transform** component stores position, rotation, and scale. Your scripts add components with your own behavior. The cube GameObject and its movement component are separate objects.

Words you will use in this lesson

Concept	Example	Meaning
Field	`private float speed = 1f;`	Stores a value inside each component instance. `float` holds a decimal number; `1f` is a float literal.
Property	`public float Speed { get ... set ... }`	Provides code with a way to read or assign a value. Its accessors can also check or calculate values.
Method	`Update()`, `Translate(...)`	A named block of work. Parentheses hold arguments when the method needs them.
Inheritance	`: MonoBehaviour`	Makes your class a kind of Unity script component. Unity can manage its lifecycle.
Composition	Cube + Transform + your three components	Several components work together on the same GameObject. Adding a component does not make the cube inherit from your class.

Read a property before you use it

```
[SerializeField] private float speed = 1f;

public float Speed
{
    get { return speed; }
    set { speed = value; }
}
```

`speed` is the private storage field. `Speed` is the public property. C# treats them as different names because capitalization matters. `get` returns the stored number; `set` stores the incoming number, called `value`.

```
// Reading examples only: mover is an existing TranslateObject component.
float currentSpeed = mover.Speed; // Runs get.
mover.Speed = 2f;                 // Runs set; value is 2f.
```

You will see how to obtain `mover` in the final experiment. The examples above are explanations, not a fourth script to create.

> **Inspector field ≠ C# property.** `[SerializeField]` lets Unity save and display the private `speed` field. The Inspector labels it "Speed." That label does not mean it is displaying the C# `Speed` property. The ordinary Inspector writes the serialized field directly, bypassing this property's setter. These examples use both so you can learn the difference. Unity serialization reference ↗

Why use a property if these accessors only read and write?

It gives other code a named way to access the value while keeping the field private. Later, a setter could reject invalid values. Here, negative speeds are allowed so you can test reverse motion. Validation in a setter alone would not validate direct Inspector edits.

How Unity runs your class

VISUAL WALKTHROUGH / ONE GAMEOBJECT

Follow the components from code to screen

Read from top to bottom. The outer box is one GameObject; the three colored cards are component instances attached to it. Use the buttons to follow the starter scripts.

`TranslateObject.cs` `RotateObject.cs` `ChangeObjectColor.cs`

↓ **Attach the scripts → Unity creates component instances**

CombinedCube

ONE GAMEOBJECT / CONTAINS THESE COMPONENTS

Transform
Position • Rotation • Scale

MeshRenderer + material
Draws the mesh using its material

YOUR THREE ATTACHED COMPONENT INSTANCES

MOVEMENT

`TranslateObject`

Private field / stores data

`speed = 1`

↓ `get` reads the field

Public C# property / get + set

`Speed`

Update() · every frame

`transform.Translate(...)`

Read Speed → calculate displacement → change world X.

ROTATION

`RotateObject`

Private field / stores data

`degreesPerSecond = 90`

↓ `get` reads the field

Public C# property / get + set

`DegreesPerSecond`

Update() · every frame

`transform.Rotate(...)`

Read DegreesPerSecond → calculate angle → change local Y rotation.

APPEARANCE

`ChangeObjectColor`

Private field / stores data

`tint = cyan`

↓ `get` reads the field

Public C# property / get + set

`Tint`

Start() · once at startup

`runtimeMaterial.color = Tint;`

Find this object's Renderer → get its material → apply Tint.

Each script inherits from `MonoBehaviour`. The GameObject **contains** these components: this is composition. The default color behavior matches the starter. The toggle below models the optional color Update exercise.

↓ **Unity calls the methods you implemented**

1 Start: initialize once

Before the first Update, the color component's `Start()` reads `Tint` and sets its material color.

2 Update: repeat each frame ↻

On enabled components of an active GameObject, Unity calls the movement and rotation `Update()` methods. They read their rate properties and multiply by `Time.deltaTime`. Their relative call order is not specified here.

3 Render: produce the image

Unity uses the resulting Transform, mesh, material, lighting, and camera to draw the object. The color stays at its startup tint while movement and rotation continue.

Follow a property write and an Inspector edit

`mover.Speed = 2f;` → calls **set** with value 2 → stores `speed = 2` → the next Update reads Speed using **get**

The optional setter exercise later in this lesson obtains `mover` with `GetComponent<TranslateObject>()`. The "Try Speed = 2" button models that code assignment; the new rate affects the next Update.

The ordinary Inspector takes a different route: **Inspector → serialized private field**. It writes `speed` directly and bypasses the C# setter. A later Speed getter still reads that field. Likewise, changing `tint` changes stored data; the starter's material keeps its applied color until code copies Tint into the material again.

Read the full path

1. The three class definitions become three component instances on one GameObject. Each instance keeps its own fields.
2. Unity calls the color instance's Start once before its first Update would run. Tint's getter supplies the saved color to the material.
3. Every frame, movement reads Speed and changes the Transform position. Rotation reads DegreesPerSecond and changes the Transform rotation. These two scripts have no Start method in the starter files.
4. The engine renders the mesh using the current Transform and material. In the starter color script there is no Update method, so its tint is applied only at startup.
5. A C# assignment to Speed calls its setter. The following movement Update reads the new field value through the getter. Unity does not call your custom properties simply because they are public.

Unity Start reference ↗ · Unity Update reference ↗ · Unity serialized fields ↗

1. **You save a .cs file.**

   Unity imports it and compiles C# into assemblies. A compiler error must be fixed before the changed script can run.

2. **You attach the script.**

   Unity creates a component instance on the selected GameObject. Its saved field values belong to that instance.

3. **You enter Play mode.**

Unity initializes scene components. For a running enabled component, `Start()` runs once before its first `Update()`.

4. **Unity runs its frame loop.**

   Unity calls `Update()` each frame on enabled components of active GameObjects. Your movement and rotation methods change the Transform.

5. **Unity renders the scene.**

   The engine uses the resulting transforms, meshes, materials, lighting, and camera to produce the image.

Unity does not read the English class name and infer movement or color. The compiled statements call engine APIs. `Start` and `Update` have specific names and signatures that Unity recognizes. `private void Update()` takes no arguments and returns no value. Your property names can differ; Unity does not automatically call arbitrary properties or methods.

For these components, you do not write a console `Main()` or create them with `new TranslateObject()`. Attach them in the Editor. Runtime code can also use `AddComponent`, which is outside this lesson.

Does Unity always update movement before rotation?

Do not rely on the order of different components' `Update()` calls. These scripts do not need a particular order: translation uses world X, while rotation changes the object's local orientation. Event function execution order ↗

03 SCENE SETUP / YOUR WORKSPACE

Prepare your scene

Use the Unity 3D project prepared in 10B, or the instructor's classroom project. Save your work before changing scenes. If Unity is unavailable, work with a team's Editor or the instructor's demonstration and record predictions and observed results. Ask the instructor how to complete any required hands-on evidence.

1. Create a new scene and save it as `Classes11A`. Keep a Main Camera and a Directional Light. Use the class project's existing 3D render pipeline.
2. In the Project window, create `Assets/Scripts/Classes11A`. Create three MonoBehaviour scripts named exactly `TranslateObject`, `RotateObject`, and `ChangeObjectColor`. Depending on your Editor, use Create → Scripting → MonoBehaviour Script, or Create → C# Script.
3. Use GameObject → 3D Object → Cube three times. Rename the cubes and set their Transform values using the table below. Do not add a Rigidbody for this Transform exercise.
4. Set the Main Camera position to `(0, 2, −10)` and rotation to `(10, 0, 0)`. Keep the cubes near the origin so they are visible in Game view.
5. Open Window → General → Console. After each script edit, save in the code editor and return to Unity. Wait for compilation; fix red errors before attaching or running.

Starting scene values

GameObject	Position (X, Y, Z)	Scale (X, Y, Z)	Component to attach
MoveCube	(−3, 0, 0)	(1, 1, 1)	TranslateObject
RotateCube	(0, 0, 0)	(1, 2, 1)	RotateObject
ColorCube	(3, 0, 0)	(1, 1, 1)	ChangeObjectColor

The taller RotateCube makes its turning easier to see. Every cube starts with rotation `(0, 0, 0)`. In Scene view, select a cube and press F to frame it.

For ColorCube, create a material named `IndicatorMaterial` and drag it onto the cube. Use **Standard** in a Built-in project, or **Universal Render Pipeline/Lit** in a URP project. Keep the material's main/base color white. Custom shaders may use different color properties.

> **Keep experiments short.** Press Play for about three seconds, observe, then stop. Moving objects can leave the camera's view. Changes to these scene components during Play normally reset when you stop. Set values outside Play mode when you want them saved in the scene.

04 CLASS 01 / POSITION

Create translation

Translation means a change in position. Replace the entire contents of `TranslateObject.cs` with this class, save, and wait for Unity to compile.

TranslateObject.cs

```
using UnityEngine;

public class TranslateObject : MonoBehaviour
{
    [SerializeField] private float speed = 1f;

    public float Speed
    {
        get { return speed; }
        set { speed = value; }
    }

    private void Update()
    {
        transform.Translate(Vector3.right * Speed * Time.deltaTime, Space.World);
    }
}
```

1. Drag the script from the Project window onto **MoveCube** in the Hierarchy. Select MoveCube and find its Translate Object component in the Inspector.
2. Set the component's Speed field to `1`. Predict the X position after about three seconds from its starting X of −3.
3. Press Play. Watch the Game view and the Transform Position in the Inspector. Stop after about three seconds. Expect X near `0`; your stopping time will vary.
4. Outside Play, set Speed to `2`, then test again. It should travel about twice as far in the same time. Try `0` for no movement and `−1` for movement toward world left.

Follow one frame

`using UnityEngine;` makes Unity types available by their short names. The class body sits between braces. `Update()` reads the `Speed` property; its getter returns `speed`. The inherited `transform` property refers to the Transform on this component's GameObject.

`Vector3.right` is `(1, 0, 0)`. Multiplying it by speed and elapsed time produces a displacement vector. `Translate(..., Space.World)` adds that displacement along the scene's axes.

At 1 unit/second and a 0.02-second frame:

```
(1, 0, 0) × 1 × 0.02 = (0.02, 0, 0)
```

Unity moves this object 0.02 units to the right for that frame.

`Time.deltaTime` is the elapsed time since the previous frame. It converts a per-second rate to this frame's displacement, so ordinary frame-rate changes do not turn speed into "units per frame."

Check your understanding

If Speed is 2 and deltaTime is 0.02, this frame adds 0.04 to world X. The script reads a property, obtains a number, calculates a displacement, then calls a Transform method. The script file remains one class definition while each attached component has its own speed.

05 CLASS 02 / ORIENTATION

Create rotation

Rotation changes which way an object faces. Create the second class in `RotateObject.cs`.

RotateObject.cs

```
using UnityEngine;

public class RotateObject : MonoBehaviour
{
    [SerializeField] private float degreesPerSecond = 90f;

    public float DegreesPerSecond
    {
        get { return degreesPerSecond; }
        set { degreesPerSecond = value; }
    }

    private void Update()
    {
        transform.Rotate(Vector3.up * DegreesPerSecond * Time.deltaTime, Space.Self);
    }
}
```

1. Save and let Unity compile. Attach the script to **RotateCube**. Keep TranslateObject off this cube for this first test.
2. Set Degrees Per Second to `90`. Predict how far it turns in one second, then run it for about one second.
3. Stop and test `45`, then `-90`. Compare the rate and direction. For an exact quarter turn, the prediction is 90 degrees after one second; manual timing is approximate.

`Vector3.up` is `(0, 1, 0)`, so the Y axis receives the angle. `Space.Self` means the object's own local axes. At 90 degrees/second and deltaTime 0.02, the frame rotates it by 1.8 degrees. Unity stores rotation internally as a quaternion; its Inspector usually shows Euler angles for editing. Values can wrap around, so 360 degrees can display as 0.

Compare the classes: both hold a rate in a field, expose a property, and use `Update()` and deltaTime. Their engine calls change different parts of the Transform. Transform.Rotate reference ↗

06 CLASS 03 / APPEARANCE

Change the color

This class changes a material's color once when the component starts. It uses a `Color` value, which contains red, green, blue, and alpha channels. The built-in value `Color.cyan` supplies a starting color.

ChangeObjectColor.cs

```
using UnityEngine;

public class ChangeObjectColor : MonoBehaviour
{
    [SerializeField] private Color tint = Color.cyan;
    private Material runtimeMaterial;

    public Color Tint
    {
        get { return tint; }
        set { tint = value; }
    }

    private void Start()
    {
        runtimeMaterial = GetComponent<Renderer>().material;
        runtimeMaterial.color = Tint;
    }

    private void OnDestroy()
    {
        if (runtimeMaterial != null)
        {
            Destroy(runtimeMaterial);
        }
    }
}
```

1. Save, let Unity compile, and attach the script to **ColorCube**, which must have its Renderer and IndicatorMaterial.
2. While stopped, open the component's Tint color picker and select a bright color. Keep alpha at 1 (or 255 if shown on a 0–255 scale).
3. Press Play. The cube should show the chosen tint, affected by the scene's lighting and material. Stop, choose another Tint, and press Play again.
4. While playing, change the component's Tint field. Predict whether the rendered cube changes immediately. It stays at the startup tint because this class applies the value only in `Start()`. Stop, set the desired Tint outside Play, and restart.

Trace the engine objects

ChangeObjectColor → Renderer → Material → color

`GetComponent<Renderer>()` finds the Renderer on the same GameObject. Angle brackets specify the type you want to find. The Renderer draws the cube's mesh. Its `material` property gives this renderer a runtime material instance, which we keep in the `runtimeMaterial` field. Assigning `runtimeMaterial.color = Tint` reads our property and writes Unity's material color property.

`Start()` runs once for this component instance before its first Update, so the code applies the tint once. A later write to Tint changes the stored data; this version needs another call to apply it to the material. Data changes produce visible results when code passes them to the engine.

Why is OnDestroy included?

Accessing `Renderer.material` can create a separate material so one renderer's tint does not change every object using the shared asset. The class stores that material and destroys it when Unity destroys this component. The `if` checks that a material exists first. This is resource cleanup; focus first on how Start finds the renderer and assigns the color. Avoid changing `sharedMaterial` for this experiment because it can alter the shared material asset. Renderer.material reference ↗

Practice: make Tint changes visible during Play

After you have tested the startup behavior, add this method *inside ChangeObjectColor's class braces*. Keep Start and OnDestroy. Now Update copies the current Tint into the material every frame, so Inspector field edits during Play become visible. Explain the extra repeated work and why Start alone was enough for a fixed startup color.

```
private void Update()
{
    runtimeMaterial.color = Tint;
}
```

Keep or remove this method based on whether the indicator should accept changing tints. This practice adds a method to the existing class; it does not add another class.

07 COMPONENTS / WORKING TOGETHER

Combine the three classes

1. Stop Play. Create another cube named **CombinedCube**, at `(0, 2, 0)` with scale `(1, 1.5, 1)`. Give it IndicatorMaterial.
2. Attach one instance of **each of the three scripts**. Set Speed to `0.5`, Degrees Per Second to `60`, and Tint to a bright color.
3. Press Play briefly. The cube should move along world X, turn around its local Y axis, and receive its startup color. Toggle off Translate Object's component checkbox during Play. Movement stops while rotation continues.
4. Stop. Duplicate CombinedCube, move the duplicate to `(−3, 2, 0)`, and set its Speed to `1.5`. Test again. Compare the two TranslateObject instances and explain how they share a class definition while holding different field values.

Use the property setter from code

Work in `RotateObject.cs`. Add the following method inside the class, alongside Update. It asks the same GameObject for its TranslateObject component and sets its Speed property. The null check lets RotateCube, which has no TranslateObject, keep working.

```
private void Start()
{
    TranslateObject mover = GetComponent<TranslateObject>();
    if (mover != null)
    {
        mover.Speed = 2f;
    }
}
```

Before Play, set CombinedCube's Inspector Speed to 0.5. Press Play and inspect it again: Start assigns 2 through the setter, which writes the speed field, so the Inspector now shows 2. The movement

component reads that value through its getter in Update. Remove this experimental Start method afterward to return control to each cube's saved speed.

Action 11A requirements / 10 points

Translation class and observed movement: 2 points. Rotation class and observed turning: 2 points. Color class and observed startup tint: 2 points. Explanation of class/instance, field/property, and Unity lifecycle using your scene: 3 points. Accessible GitHub repository URL submitted in Canvas the same day: 1 point.

Explain using your own scene

- Identify one class definition and two instances of it. Which data belongs to each instance?
- Point to a field, property, getter, setter, and method in your scripts. Explain what each does.
- Trace a frame from Update → Speed getter → Translate → changed Transform → rendered image.
- Explain why the first color class does not react immediately to Tint edits during Play.
- Explain why three components on a GameObject are composition, and where inheritance appears in the code.

Keep the three .cs files, your saved Unity scene, and brief before/after observations with your existing project work. A useful observation includes the starting value, your prediction, the actual result, and the line that caused it. Commit the three scripts and your saved scene with their Unity .meta files to your existing project repository. Include brief README observations and screenshots showing the three behaviors and the combined object; reuse evidence from the tests above. Push, check instructor access, and submit the GitHub repository URL to Action 11A in Canvas before lab ends the same day. Keep Library, Temp, Logs, and other generated Unity folders out of Git. If setup blocks your run, record the exact error and ask the instructor during class about an approved evidence alternative.

> **Ready to explain?** Ask a partner to choose a speed, rotation rate, or tint. Predict the result, run the scene, then point to the class and engine call responsible. Use the result to find any part you still need to study.

08 SUPPORT / TROUBLESHOOTING

Help & references

Find the cause before changing more code

What you see	What to check
Script will not attach	Fix all Console compiler errors. Match file and class names exactly. Keep `: MonoBehaviour` and check braces.
No movement or rotation	Press Play; check the GameObject is active, its component is enabled, and its rate is not zero. Select the actual cube, rather than the script asset.
Cube leaves the view	Stop Play. Reset its starting position and lower Speed. Select it in Scene view and press F.
A new code default does not change Inspector Speed	The attached component retains its saved field value. Edit Speed on that instance in the Inspector while stopped.
NullReferenceException in color Start	Attach the color script to the cube with the Renderer. An empty parent object has no Renderer of its own. Verify that a material is assigned.
Pink or unchanged color	Use a material/shader supported by your project's pipeline. Check Tint outside Play, press Play again, and keep the material base color white. The starter applies color only once.
New values disappear after Stop	Set this scene component's desired values outside Play, then save the scene.

When asking for help, include the class, the relevant line, what you expected, what happened, and the exact Console error. You may use your team, instructor, or an AI explanation to investigate. Verify an explanation against your code and official documentation; no paid or personal AI account is required for this lesson.

Official references

- Microsoft: C# classes ↗ · C# properties and accessors ↗
- Unity: Update ↗ · Lifecycle and execution order ↗

- Transform.Translate ↗ · Transform.Rotate ↗ · Time.deltaTime ↗
- Serialized fields and properties ↗ · Material.color ↗

Return to CS077 in Canvas ↗